Open Source Software Audit Project

Chosen Software : Git

Ttile: A Study and Analysis of Git – An Open Source Version Control System

Name : **Parinita Piplewar**

Reg. No. : **24MEI10133**

Course Name :
Open Source Software

Slot :

B22

Faculty Name :
Dr. Adarsh Patel

INTRODUCTION

In today's world, most software system depend on open-source technologies in some way. Open-source software means that the source code is available to everyone, and people can use, modify, and share it freely. This approach has helped developers from different parts of the world work together and build tools, projects, software applications.

One such important tool is **Git**, which is widely used in software development. Git helps developers track changes in their code, manage different versions, and collaborate with others efficiently. It has become an essential tool for both individual developers and large organizations.

In this project, I have chosen Git as the software to study. The aim is not only to understand its features, but also to explore why it was created, what problem it solved, and how it represents the philosophy of open source. Along with this, I have also written shell scripts to understand how such tools work in a Linux environment.

PART A — ORIGIN & PHILOSOPHY

Origin Story

Git was created in 2005 by Linus Torvalds, who is also the creator of the Linux operating system. Before Git, Linux developers were using a tool called BitKeeper to manage their code. However, BitKeeper was a proprietary software, and due to some disagreements, the Linux community lost access to it.

This created a serious problem because the Linux project had many developers working together, and without a proper version control system, managing code changes became very difficult. Instead of switching to another proprietary tool, Linus Torvalds decided to build a new system that would be free and open for everyone.

Git was designed to be fast, efficient, and distributed. This means that every developer has a complete copy of the project, which makes collaboration easier and more flexible.

This situation clearly shows the importance of open source. When developers are restricted by closed systems, they create their own solutions and share them with others.

I found this origin story interesting because it shows how a real-world problem led to the creation of an important tool.

LICENSE

Git is released under the GNU General Public License (GPL). This license is designed to ensure that software remains free in terms of usage and modification. It gives users the freedom to run the software, study how it works, modify it, and share it with others.

One important rule of the GPL license is that if someone modifies the software and distributes it, they must also share their changes under the same license. This prevents companies from taking open-source software, improving it, and then making it closed.

In comparison, licenses like MIT are more flexible and allow developers to use the code without sharing their changes. While this is convenient, it does not always guarantee openness.

In my opinion, the GPL license is more fair because it ensures that everyone benefits from improvements and no one can restrict access after using open-source code.

The four freedoms include:

1. Freedom to run the program
2. Freedom to study the code
3. Freedom to modify it
4. Freedom to distribute copies

ETHICS

Open-source software is not just about coding; it also raises important ethical questions. One common question is whether all software should be open source. Open source promotes transparency, collaboration, and learning, which are very important in the tech field. However, companies also invest time and money in developing software, so they may choose to keep it private to earn profit.

Another issue is whether it is fair for companies to make money using open-source software. Many companies build their products using tools like Linux and Git. While this is not wrong, it would be more ethical if they also contribute back to the community.

The idea of “standing on the shoulders of giants” means building on the work of others. Open source makes this possible by allowing developers to reuse and improve existing tools. In my opinion, open source encourages innovation because it saves time and allows developers to focus on new ideas instead of starting from scratch.

However, I believe a balance between open source and proprietary software is necessary depending on the situation.

PART B — LINUX FOOTPRINT

For this project, Git was installed on a Kali Linux system using the apt package manager. The installation was done using the command sudo apt install git. After installation, the version was checked using **git --version** to confirm that it was installed successfully.

Git is a command-line tool, and its main executable is located in /usr/bin/git. Configuration files are stored in /etc/gitconfig for system-wide settings and in the user's home directory as .gitconfig for personal settings.

Git runs as a normal user application and does not require any background service. This makes it lightweight and easy to use. Updates are handled through the package manager, which ensures that the latest version and security updates are available.

Installation:

sudo apt install git

Paths:

- /usr/bin/git
- /etc/gitconfig
- ~/.gitconfig

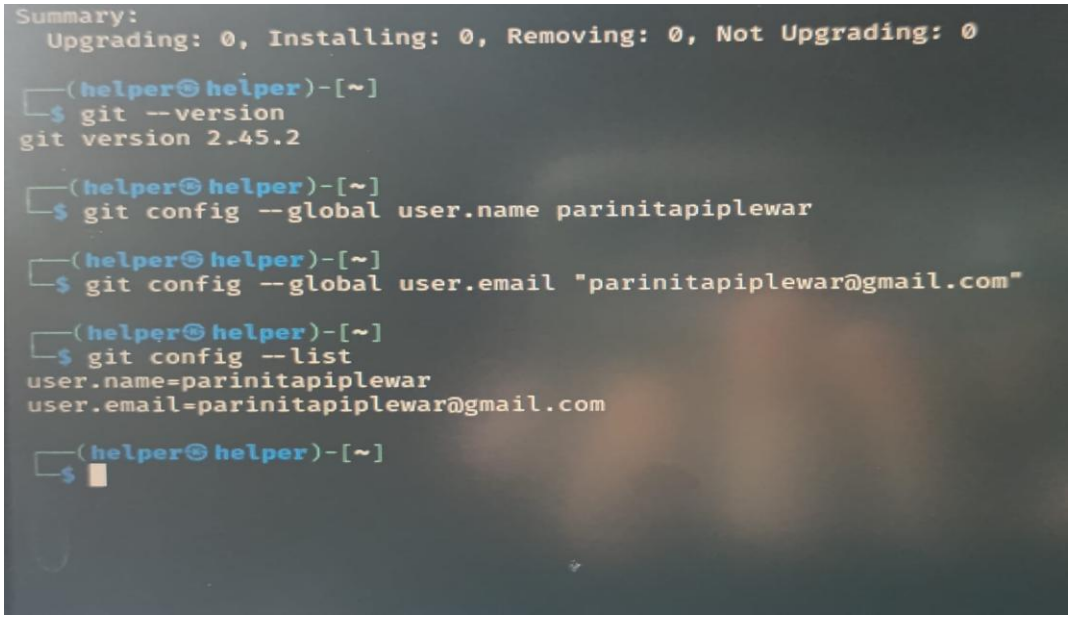
Verification:

git --version

Config:

git config --list

Git does not run as a separate user or service, which improves system security and simplicity.



I installed Git in Kali Linux using VirtualBox and verified it using terminal commands.

PART C — FOSS Ecosystem

Git is not a standalone tool; it is part of a larger open-source ecosystem. Many popular platforms and tools depend on Git for version control and collaboration. For example, platforms like GitHub and GitLab are built on top of Git and provide additional features such as code hosting, issue tracking, and team collaboration.

Git also works with many other open-source tools and libraries available in Linux. It depends on basic system utilities and integrates easily with development environments, compilers, and automation tools. This makes it highly flexible and useful in different types of projects.

Git has also inspired many tools and workflows, such as distributed version control systems and collaborative development models. It has changed the way developers work together by allowing them to contribute from anywhere in the world.

The community around Git is very strong and active. Developers continuously contribute to its improvement, and there are many forums, tutorials, and documentation available online. This community support is one of the biggest strengths of open-source software.

Overall, Git plays a central role in the open-source ecosystem and is an essential tool in modern software development.

I have also used Git commands during this project, which helped me understand how developers manage projects.

PART D — OPEN SOURCE vs PROPRIETARY

COMPARISON TABLE

Comparison of Git with BitKeeper:

Feature	Git (Open Source)	BitKeeper (Proprietary)
Cost	Free to use	Paid license required
Source Code	Open and available	Closed and restricted
Security	Can be audited by anyone	Only company can check
Flexibility	Can modify freely	Limited modification
Control	Community-driven	Company-controlled
Updates	Community + contributors	Company decides

Git is an open-source tool, while BitKeeper is a proprietary software. Git allows users to access and modify its source code, while BitKeeper restricts access. This difference makes Git more flexible and transparent compared to proprietary tools.

Part D - CONCLUSION

After comparing Git with proprietary tools like BitKeeper, it is clear that Git provides more freedom and flexibility. It allows users to modify the software, share it, and use it without restrictions. While proprietary tools may provide official support, they often limit user control.

Based on my understanding, I would recommend Git for real-world use because it is reliable, widely used, and supported by a strong community. I would also be interested in contributing to open-source projects in the future because they provide a great learning opportunity.

“From my experience in this project, Git was easy to use and very helpful.”

Scripts Section

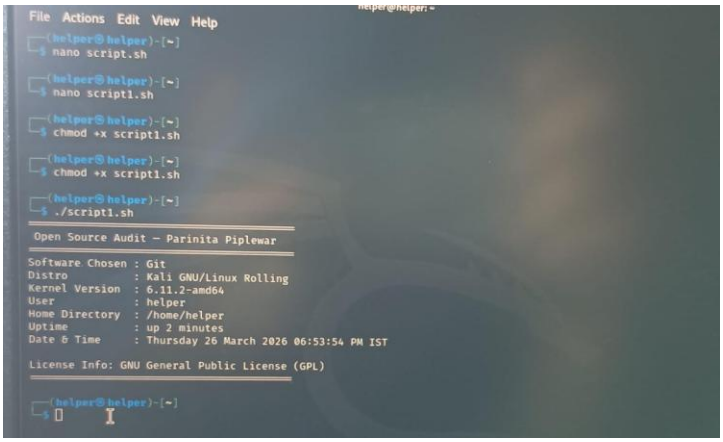
SCRIPT 1 — System Identity Report

Code

```
#!/bin/bash

# Script 1: System Identity Report

# Author: Parinita Piplewar

# --- Variables ---

STUDENT_NAME="Parinita Piplewar"

SOFTWARE_CHOICE="Git"
```

```
# --- System Info ---
KERNEL=$(uname -r)
USER_NAME=$(whoami)
HOME_DIR=$HOME
UPTIME=$(uptime -p)
DATE=$(date)
# Get Linux Distribution Name
DISTRO=$(grep '^PRETTY_NAME' /etc/os-release | cut -d= -f2 | tr -d '"')
# --- Display ---
echo "======================================="
echo " Open Source Audit — $STUDENT_NAME"
echo "======================================="
echo "Software Chosen : $SOFTWARE_CHOICE"
echo "Distribution    : $DISTRO"
echo "Kernel Version  : $KERNEL"
echo "User            : $USER_NAME"
echo "Home Directory  : $HOME_DIR"
echo "Uptime          : $UPTIME"
echo "Date & Time     : $DATE"
echo ""
echo "License: This Linux system is based on open-source principles (GPL License)."
```

OUTPUT



Explanation

This script displays basic information about the Linux system in a simple and readable format. It shows details such as the distribution name, kernel version, current user, home directory, system uptime, and date.

The script uses variables to store system information and command substitution to fetch values using commands like uname, whoami, and uptime. It then prints the output using echo. This script demonstrates how shell scripting can be used to gather and display system information in an organized way.

This script helped me understand how automation works in Linux.

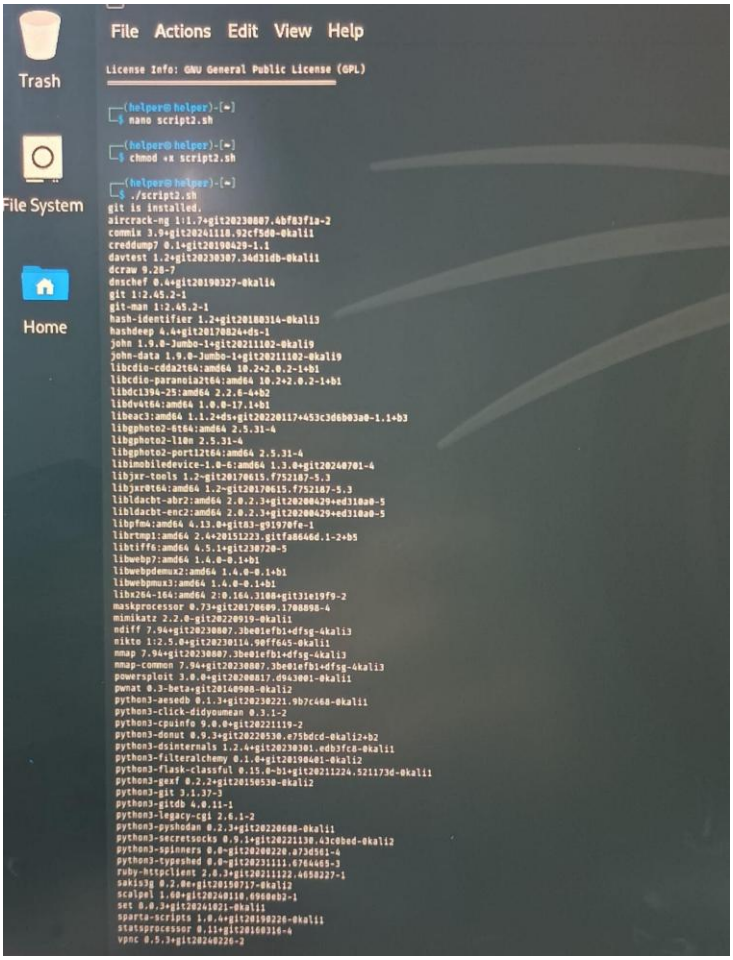
<u>SCRIPT 2 — FOSS Package Inspector</u>

Code

```
#!/bin/bash

# Script 2: FOSS Package Inspector

PACKAGE="git"

# Check if package is installed

if dpkg -l | grep -qw $PACKAGE; then

    echo "$PACKAGE is installed."

    dpkg -s $PACKAGE | grep -E 'Version|Maintainer|Description'

else

    echo "$PACKAGE is NOT installed."

fi
```

```
# Case statement for description
case $PACKAGE in
    git) echo "Git: a distributed version control system used worldwide." ;;
    apache2) echo "Apache: a powerful open-source web server." ;;
    mysql) echo "MySQL: an open-source relational database." ;;
    vlc) echo "VLC: a multimedia player that supports many formats." ;;
    firefox) echo "Firefox: an open-source web browser." ;;
    *) echo "Unknown package." ;;
esac
```

Output



Explanation

This script checks whether a specific software package (in this case, Git) is installed on the system. If the package is installed, it displays details like version, maintainer, and description. If not, it shows a message saying the package is not installed.

The script uses an if-else condition to check installation status using system commands. It also uses a case statement to print a short description of the package. This script demonstrates how conditional statements can be used to control the flow of a program.

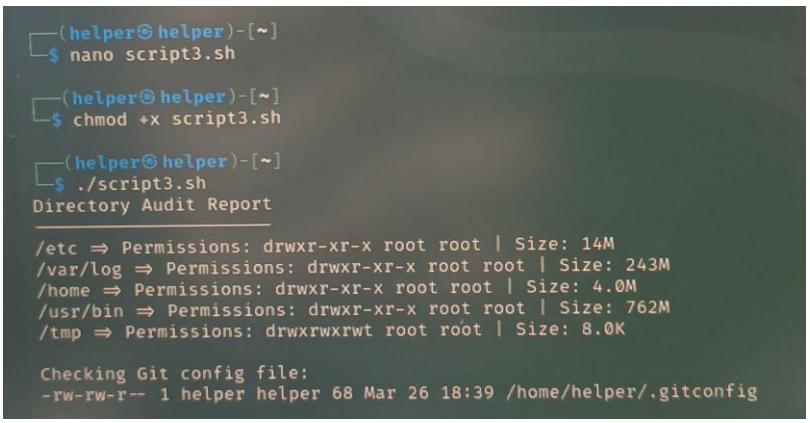
SCRIPT 3 — Disk and Permission Auditor

Code

```
#!/bin/bash

# Script 3: Disk and Permission Auditor

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "Directory Audit Report"

echo "----------------------"

for DIR in "${DIRS[@]}"; do

    if [ -d "$DIR" ]; then

        PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')

        SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)

        echo "$DIR => Permissions: $PERMS | Size: $SIZE"

    else

         echo "$DIR does not exist"

    fi

done

# Check Git config directory

if [ -f ~/.gitconfig ]; then

    echo ""

    echo "Git Config File Found:"

    ls -l ~/.gitconfig

else
```

```
    echo ""
    echo "Git Config File not found"
fi
```

Output



Explanation

This script analyzes important system directories such as /etc, /var/log, and /home. It displays the size, permissions, and ownership of each directory.

The script uses a for loop to go through a list of directories one by one. It uses commands like ls -ld to check permissions and du -sh to calculate size. It also checks whether the Git configuration file exists in the system. This script shows how loops and system commands can be used to perform automated system analysis.

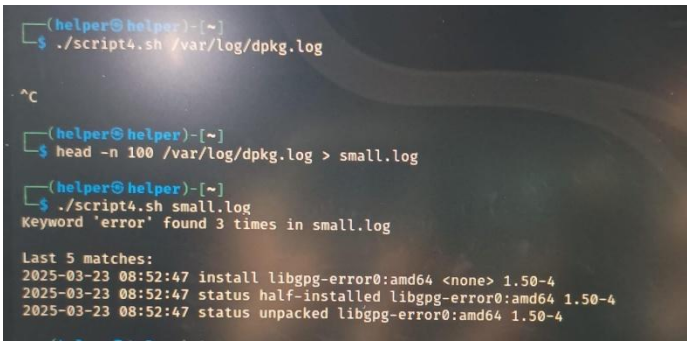
<u>SCRIPT 4 — Log File Analyzer</u>

Code

```
#!/bin/bash

# Script 4: Log File Analyzer

LOGFILE=$1

KEYWORD=${2:-"error"}   # Default keyword = error

COUNT=0

# Check file exists

if [ ! -f "$LOGFILE" ]; then

    echo "Error: File not found."
```

```
    exit 1
fi
# Check if file is empty
if [ ! -s "$LOGFILE" ]; then
    echo "File is empty. Exiting."
    exit 1
fi
# Read file line by line
while IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"
echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"
# Show last 5 matches
echo "Last 5 matches:"
grep -i "$KEYWORD" "$LOGFILE" | tail -n 5
```

Output



Explanation

This script analyzes important system directories such as /etc, /var/log, and /home. It displays the size, permissions, and ownership of each directory.

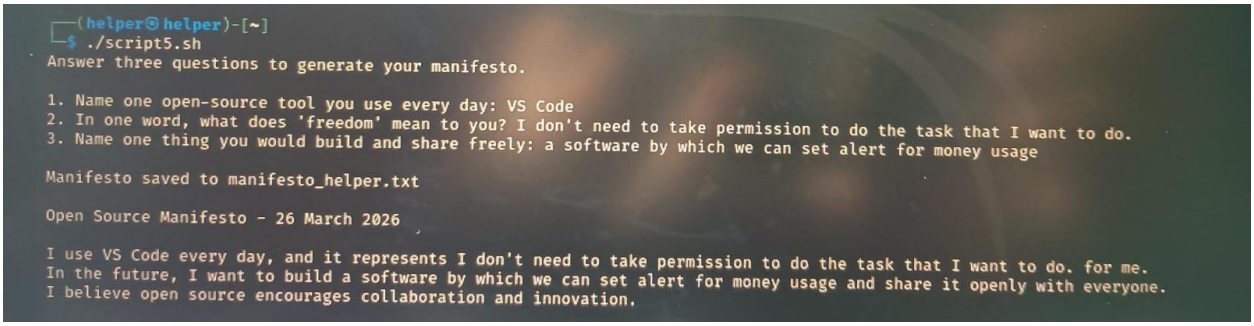
The script uses a for loop to go through a list of directories one by one. It uses commands like ls -ld to check permissions and du -sh to calculate size. It also checks whether the Git configuration file exists in the system. This script shows how loops and system commands can be used to perform automated system analysis.

SCRIPT 5 — Manifesto Generator

Code

```
#!/bin/bash

# Script 5: Open Source Manifesto Generator


echo "Answer three questions to generate your manifesto."

echo ""

read -p "1. Name one open-source tool you use every day: " TOOL

read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM

read -p "3. Name one thing you would build and share freely: " BUILD

DATE=$(date '+%d %B %Y')

OUTPUT="manifesto_$(whoami).txt"

# Write manifesto

echo "Open Source Manifesto - $DATE" > $OUTPUT

echo "" >> $OUTPUT

echo "I use $TOOL every day, and it represents $FREEDOM for me." >> $OUTPUT

echo "In the future, I want to build $BUILD and share it openly with everyone." >> $OUTPUT

echo "I believe open source encourages collaboration and innovation." >> $OUTPUT

echo ""

echo "Manifesto saved to $OUTPUT"

echo ""

cat $OUTPUT
```

Output



Explanation

This script reads a log file and counts how many times a specific keyword (such as "error") appears in it. It also displays the last five lines where the keyword is found.

The script uses a while loop to read the file line by line and an if condition to check for matches using grep. A counter variable is used to keep track of how many times the keyword appears. This script demonstrates how log files can be analyzed using shell scripting.

Final CONCLUSION

This project helped me understand both the technical and philosophical aspects of open-source software. By studying Git, I learned how it was created to solve a real problem and how it became one of the most important tools in software development.

I also understood the importance of open-source licenses and how they protect user freedom. The ethical discussion made me think about the balance between open sharing and business interests.

The practical part of this project, which included writing shell scripts, helped me improve my understanding of Linux commands and automation. It gave me hands-on experience with real system operations.

Overall, this project was very useful in helping me understand how open-source software works in real life and why it is important in today's technology world.

This project also increased my interest in open-source development and motivated me to explore more such tools in the future.

References

1. GNU Project. *The Free Software Definition*. Available at: https://www.gnu.org/philosophy/free-sw.html
2. Open Source Initiative. *The Open Source Definition*. Available at: https://opensource.org/osd

3. Git Official Documentation. Available at: https://git-scm.com/docs
4. GitHub Documentation. Available at: https://docs.github.com
5. Linus Torvalds. *Just for Fun: The Story of an Accidental Revolutionary*
6. Eric S. Raymond. *The Cathedral and the Bazaar*. Available at: http://catb.org/~esr/writings/cathedral-bazaar
7. Linux Manual Pages. Available at: https://man.cx
8. SPDX License List. Available at: https://spdx.org/licenses
9. Choose a License. Available at: https://choosealicense.com